

Assignment 3: Views, Triggers, and Data Access Design

Part 1: User Access Design

Admins

Admins are internal platform operators who monitor the system, handle disputes, and manage data across the platform. They need full read access to all tables including CUSTOMER, ORDERS, RESTAURANT, DRIVER, DRIVER_LOCATION, MENU_ITEM, ORDER_ITEM, PROMOTION, and REVIEW. They also need write access for tasks like adding or removing restaurants, suspending drivers, deleting fake reviews, and creating or expiring promotions. One important restriction is that completed order totals should not be changed after settlement, and driver location history should stay unchanged since it acts as an audit trail. Actions taken by admins on sensitive data should also be logged.

Restaurant Owners

Restaurant owners should only have access to data related to their own restaurant. They need to see orders placed at their restaurant, including items, quantities, prices, order time, and status, so they can prepare orders. They should not be able to see customer emails, phone numbers, or full profiles. They should be able to update menu item availability and view reviews for their restaurant. They also should not have access to data from other restaurants, driver GPS history, promotion details, or any unrelated customer data.

Customers

Customers are the end users who place and track orders. They should be able to view their own order history, including restaurant name, items, prices, and order status, since this information is needed for receipts and disputes. During an active delivery, they can see the driver's first name and vehicle type, but not sensitive details like phone number, license plate, or exact location. Customers should never see other customers' data or internal system data. They can submit reviews for completed orders, but they should not be able to edit or delete them afterward.

Part 2: Views

View 1 - vw_admin_order_summary (Admin)

What it does:

This view joins ORDERS, CUSTOMER, RESTAURANT, and DRIVER to create one row per order. It includes order ID, time, delivery time, status, total price, customer info, restaurant name, and driver name. This makes it easier for admins to quickly review orders or investigate issues without writing complex queries each time.

Why this design:

Instead of having admins repeatedly write the same joins, this view keeps everything consistent and easier to use. If something changes in the database later, only the view needs to be updated instead of every query.

```
CREATE OR REPLACE VIEW vw_admin_order_summary AS
SELECT
    o.order_id,
    o.order_time,
    o.delivery_time,
    o.status,
    o.total_price,
    c.name      AS customer_name,
    c.email     AS customer_email,
    r.name      AS restaurant_name,
    d.name      AS driver_name
FROM ORDERS o
JOIN CUSTOMER c ON o.customer_id = c.customer_id
JOIN RESTAURANT r ON o.restaurant_id = r.restaurant_id
LEFT JOIN DRIVER d ON o.driver_id = d.driver_id;
```

View 2 - vw_restaurant_orders (Restaurant Owner)

What it does:

This view joins ORDERS, ORDER_ITEM, and MENU_ITEM to show one row per item in an order. It includes order ID, time, status, restaurant ID, item name, quantity, and price. Restaurant owners can filter this by their restaurant_id. It does not include sensitive customer information.

Why this design:

If restaurant owners had access to full tables, there would be a risk of exposing customer data. By limiting the data in the view itself, the database helps enforce privacy instead of relying only on the application.

```

CREATE OR REPLACE VIEW vw_restaurant_orders AS
SELECT
    o.order_id,
    o.order_time,
    o.status,
    o.restaurant_id,
    mi.name AS item_name,
    oi.quantity,
    oi.unit_price
FROM ORDERS o
JOIN ORDER_ITEM oi ON o.order_id = oi.order_id
JOIN MENU_ITEM mi ON oi.item_id = mi.item_id;

-- Usage: SELECT * FROM vw_restaurant_orders WHERE restaurant_id = ?;

```

View 3 - vw_customer_order_history (Customer)

What it does:

This view joins ORDERS, ORDER_ITEM, MENU_ITEM, and RESTAURANT to show a customer's order history. It includes restaurant name, order date, item name, quantity, price, and status. Customers filter it using their customer_id.

Why this design:

This view uses ORDER_ITEM.unit_price instead of MENU_ITEM.price because menu prices can change. This ensures that past orders always reflect what the customer actually paid.

```

CREATE OR REPLACE VIEW vw_customer_order_history AS
SELECT
    o.customer_id,
    o.order_id,
    o.order_time,
    o.status,
    r.name AS restaurant_name,
    mi.name AS item_name,
    oi.quantity,
    oi.unit_price
FROM ORDERS o
JOIN ORDER_ITEM oi ON o.order_id = oi.order_id
JOIN MENU_ITEM mi ON oi.item_id = mi.item_id
JOIN RESTAURANT r ON o.restaurant_id = r.restaurant_id;

-- Usage: SELECT * FROM vw_customer_order_history WHERE customer_id = ?;

```

Part 3: Triggers

Trigger 1 - trg_prevent_invalid_status_transition

Event: BEFORE UPDATE on ORDERS

What it does:

This trigger makes sure that order statuses follow the correct progression. The valid flow is pending to confirmed to preparing to out_for_delivery to completed, or any state to cancelled. It prevents invalid or out-of-order updates.

What breaks without it:

Without this, an order could be set back to an earlier status, which could cause duplicate deliveries, incorrect billing, or inconsistent data.

```
DELIMITER $$

CREATE TRIGGER trg_prevent_invalid_status_transition
BEFORE UPDATE ON ORDERS
FOR EACH ROW
BEGIN
    IF NOT (
        (OLD.status = 'pending'      AND NEW.status IN ('confirmed', 'cancelled')) OR
        (OLD.status = 'confirmed'    AND NEW.status IN ('preparing', 'cancelled')) OR
        (OLD.status = 'preparing'    AND NEW.status IN ('out_for_delivery', 'cancelled')) OR
        (OLD.status = 'out_for_delivery' AND NEW.status IN ('completed', 'cancelled')) OR
        (OLD.status = NEW.status)
    ) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Invalid order status transition.';
    END IF;
END$$

DELIMITER ;
```

Trigger 2 - trg_recalculate_order_total

Event: AFTER INSERT on ORDER_ITEM

What it does:

This trigger updates the total_price in ORDERS whenever a new item is added. It calculates the total based on all items in the order so the value stays accurate.

What breaks without it:

If the application forgets to update the total, the stored value could become incorrect, leading to wrong receipts or reporting issues.

```

DELIMITER $$

CREATE TRIGGER trg_recalculate_order_total
AFTER INSERT ON ORDER_ITEM
FOR EACH ROW
BEGIN
    UPDATE ORDERS
    SET total_price = (
        SELECT SUM(quantity * unit_price)
        FROM ORDER_ITEM
        WHERE order_id = NEW.order_id
    )
    WHERE order_id = NEW.order_id;
END$$

DELIMITER ;

```

Trigger 3 - trg_log_order_status_change

Event: AFTER UPDATE on ORDERS

What it does:

This trigger logs every status change into ORDER_STATUS_LOG, including the old status, new status, and timestamp. This helps track what happened to an order over time.

What breaks without it:

Without this log, there would be no history of status changes, making it harder to investigate issues or disputes.

-- Prerequisite table (run before creating the trigger)

```

CREATE TABLE IF NOT EXISTS ORDER_STATUS_LOG (
    log_id      INT AUTO_INCREMENT PRIMARY KEY,
    order_id    INT                NOT NULL,
    old_status  VARCHAR(30)       NOT NULL,
    new_status  VARCHAR(30)       NOT NULL,
    changed_at  DATETIME          NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (order_id) REFERENCES ORDERS(order_id)
);

```

```

DELIMITER $$

CREATE TRIGGER trg_log_order_status_change
AFTER UPDATE ON ORDERS
FOR EACH ROW
BEGIN
    IF OLD.status <> NEW.status THEN
        INSERT INTO ORDER_STATUS_LOG (order_id, old_status, new_status)
        VALUES (NEW.order_id, OLD.status, NEW.status);
    END IF;
END$$

DELIMITER ;

```

Part 4: Reflection

Hardest Design Decision

The hardest part was handling restaurant reviews. In Assignment 2, REVIEW is linked to CUSTOMER but not directly connected to RESTAURANT. The only way to connect them is through ORDERS, which can be unclear if a customer has ordered from multiple restaurants. Looking back, it would have been better to include a restaurant_id or order_id in REVIEW to make this relationship clearer.

Did Building Views and Triggers Change How You Think About the Schema?

Yes, it did. Writing the status trigger made it clear that ORDERS.status is just a VARCHAR with no restriction, so invalid values could still be inserted. Adding a CHECK constraint or ENUM would improve this.

I also realized that storing total_price creates extra work to keep it consistent. If it were calculated when needed instead, the trigger would not be necessary.

One Thing Triggers Cannot Protect Against

Triggers cannot fully handle race conditions. If two updates happen at the same time, both might pass the checks and still cause conflicts. Fixing this would require something like locking or application-level control, which triggers alone cannot enforce.